Universidade de São Paulo
Instituto de Matemática,
Estatística e Ciência da Computação
Bacharelado em Ciência da Computação

Aplicações de Computação Gráfica na construção de visuais estilizados, level design e jogabilidade para um jogo 3D

Diogo Ribeiro de Faria, Adriano Elias Andrade,
Guilherme Henrique Vasconcelos da Cunha

Monografia Final

mac 499 — Trabalho de
Formatura Supervisionado

Supervisor: Dr. Wilson Kazuo Mizutani

São Paulo
2026

O conteúdo deste trabalho é publicado sob a licença CC BY 4.0
(Creative Commons Attribution 4.0 International License)

Agradecimentos

Do. Or do not. There is no try.
— Mestre Yoda

Texto texto texto

Resumo

Diogo Ribeiro de Faria, Adriano Elias Andrade, Guilherme Henrique Vasconcelos da Cunha. **Aplicações de Computação Gráfica na construção de visuais estilizados, level design e jogabilidade para um jogo 3D**. Monografia (Bacharelado). Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, em forma de texto. Deve apresentar os objetivos, métodos empregados, resultados e conclusões. O resumo deve ser redigido em parágrafo único, conter no máximo 500 palavras e ser seguido dos termos representativos do conteúdo do trabalho (palavras-chave). Deve ser precedido da referência do documento. Texto texto texto

Abstract

Diogo Ribeiro de Faria, Adriano Elias Andrade, Guilherme Henrique Vasconcelos da Cunha. **Applications of Computer Graphics in the creation of stylized visuals, level design, and gameplay for a 3D game**. Capstone Project Report (Bachelor). Institute of Mathematics, Statistics, and Computer Science, University of São Paulo, São Paulo, 2026.

Elemento obrigatório, elaborado com as mesmas características do resumo em língua portuguesa. De acordo com o Regimento da Pós-Graduação da USP (Artigo 99), deve ser redigido em inglês para fins de divulgação. É uma boa ideia usar o sítio www.grammarly.com na preparação de textos em inglês. Text text text

Sumário

1 Introdução **1**
Objetivos . . . 2
Estrutura do Texto . . . 3

2 Conceitos **5**
2.1 Isometria . . . 5
2.2 *Low Poly* . . . 10
2.3 *Fast-Paced* . . . 13

3 Metodologia **15**
3.1 Projeto . . . 15
3.2 Desenvolvimento . . . 16
3.3 Arquitetura . . . 16
3.4 Modelagem . . . 17
3.5 *Shaders* . . . 18

4 Proposta **19**
4.1 Câmera . . . 19
4.2 Movimentação . . . 19
4.3 Progressão e Mecânicas de Jogo . . . 20
4.3.1 Fase 1 - Montanha (Mobilidade e Defesa) . . . 20
4.3.2 Fase 2 - Caverna (Interação Física e Ataque) . . . 20
4.3.3 Fase 3 - Floresta (Controle Tático e Quebra-cabeças) . . . 21
4.4 Efeitos Visuais . . . 21
4.5 Pós-processamento . . . 21
4.6 Interação do Usuário . . . 21
4.7 Som . . . 21
4.8 Configurações . . . 21

5 Entidades 23

6 Planos Futuros 25

7 Conclusão 27

Referências 29

Capítulo 1

Introdução

Em 20 de Abril de 1951 começa oficialmente a operação do *Whirlwind I*, o primeiro computador a conseguir exibir imagens gráficas em um monitor de tubo de raios catódicos em tempo real, permitindo interação com o usuário. Desenvolvido por Jay Forrester e Robert Everett no MIT (*Massachusetts Institute of Technology*) e financiado pelas Forças Armadas dos Estados Unidos, este computador tinha como objetivo inicial ser um simulador de voo analógico, mas que eventualmente evoluiu para um sistema eletrônico digital. Embora desenvolvido para este propósito, este computador já contava com um dos primeiros jogos eletrônicos da história, o *bouncing ball game*, feito para demonstrar a simulação física de movimento de uma bola quicando.

Desde então, a computação gráfica e os jogos evoluíram lado a lado, impulsionados por uma busca contínua pelo fotorrealismo e por uma simulação fiel do mundo real. Essa corrida tecnológica existe até os dias de hoje, com o exemplo recente mais famoso sendo a implementação do DLSS5 em placas de vídeo modernas, uma tecnologia que utiliza redes neurais e algoritmos de aceleração baseados em Inteligência Artificial para renderizar iluminação em tempo real e cenas geometricamente complexas.

No entanto, para atingir este nível de processamento, a computação gráfica foi limitada pelas restrições de hardware ao decorrer da história. Nas décadas de 1980 e 1990, a incapacidade de processar grandes volumes de dados em tempo real exigia soluções criativas de otimização, sendo então consolidadas técnicas como a modelagem de baixa poligonização (*low-poly*), para uma renderização mais eficiente, e a projeção isométrica, utilizada para simular profundidade e exibir grandes cenários tridimensionais em ambientes 2D com baixo custo de processamento.

Com o avanço tecnológico, técnicas como essas deixaram de ser requisitos técnicos e se tornaram uma linguagem de expressão visual. Enquanto jogos de alto orçamento (*AAA*) continuam perseguindo a complexidade gráfica, desenvolvedores independentes conseguem usar desta linguagem para liberar recursos de hardware e focar em outros aspectos de seus jogos, como um desempenho mais acessível para o público e em como os aspectos visuais podem afetar o *game design* e o *level design* do projeto.

Nesse quesito, o estilo do jogo influencia diretamente o *game design* — conjunto de

regras, mecânicas e dinâmicas de interação — e o *level design*, responsável pelo planejamento espacial, arquitetura dos ambientes e pela curva de desafio oferecida ao jogador. Para este projeto, destaca-se em particular jogos das categorias de aventura e *fast-paced*. Enquanto jogos de aventura exigem cenários visualmente desenvolvidos que estimulem a exploração, a navegação e a resolução de quebra-cabeças, jogos *fast-paced* demandam uma movimentação dinâmica e precisa, com respostas imediatas aos comandos do jogador.

Portanto, com a redução de detalhes em modelos *low-poly* e a projeção isométrica da câmera do jogo, é possível manipular como o jogador identifica obstáculos, trajetórias e diferentes geometrias de cenário enquanto executa comandos, navega e enfrenta diferentes desafios.

Então, sob a perspectiva deste trabalho, a escolha destas técnicas permite a exploração das etapas do *pipeline* gráfico para aprofundar o conhecimento em **Aplicações de Computação Gráfica na construção de visuais estilizados, *level design* e jogabilidade para um jogo 3D**, por meio da implementação de diferentes interações entre objetos e métodos de renderização de modelos, pixels e iluminação.

Objetivos

O objetivo deste trabalho é projetar, desenvolver e documentar um jogo eletrônico de aventura e ação *fast-paced* em perspectiva isométrica 3D, utilizando estética *low-poly* pixelizada, com foco na integração entre mecânicas de física, computação gráfica, *game design* e *level design*.

Para atingir este objetivo, foram estabelecidas as seguintes metas:

Desenvolver a arquitetura básica de movimentação e colisão: Implementar um sistema de movimentação dinâmico e responsivo (*fast-paced*) focado em física, incluindo mecânicas de aceleração, deslize em superfícies e navegação fluida em rampas e obstáculos;

Estruturar o primeiro cenário (Movimentação): Projetar desafios de *level design* baseados em navegação avançada e mecânicas de mobilidade (*parkour*);

Estruturar o segundo cenário (Interação): Implementar sistemas de manipulação e destruição de objetos tridimensionais do cenário, permitindo a abertura de novas rotas e resolução de obstáculos;

Estruturar o terceiro cenário (Iluminação): Desenvolver quebra-cabeças e dinâmicas de ambiente fundamentados em técnicas de iluminação e reatividade de materiais à luz;

Implementar o confronto final e integração de mecânicas: Projetar um combate contra um chefe final que exija a aplicação combinada de todas as ferramentas e mecânicas adquiridas ao longo dos cenários anteriores;

Aplicar técnicas de estilização no *pipeline* gráfico: Integrar os modelos *low-poly*, a renderização pixelizada e a projeção ortográfica isométrica, mantendo a legibilidade espacial e o desempenho do jogo.

Estrutura do Texto

Além desta introdução, este documento está organizado em outros 20 capítulos, estruturados da seguinte forma:

Inspirações: Relata as mecânicas e referências estudadas de outros jogos e como afetam o desenvolvimento do projeto.

Organização: Descreve os *softwares* e sistemas de gestão utilizados para o desenvolvimento do trabalho, incluindo estilo de documentação de código e *commits*.

Movimentação: Explica a etapa de projeção vetorial realizada para o movimento da câmera e do jogador para seguir precisamente o comando do usuário, e o sistema de navegação dos inimigos.

Tampa, Tesoura e Vagalume: Explica os diferentes itens implementados no jogo e a forma como o usuário pode utilizá-los para interagir com o mapa, inimigos, ou integrá-los durante sua exploração.

Modelos: Descreve a etapa para construção e animações dos modelos do cenário, entidades e itens, além dos *softwares* utilizados.

Entidades: Explica a implementação dos objetos instanciáveis do jogo, incluindo inimigos, projéteis, figurantes e o jogador, detalhando a forma como são controlados e como interagem com o cenário.

Montanha, Floresta, Caverna, Covil e Vilarejo: Relata o processo criativo para o design de cada fase, suas características visuais, conteúdo, desafios de desenvolvimento e coesão de continuidade.

Efeitos Visuais: Descreve como foi implementado o *feedback* visual que guia e mostra ao jogador as implicações de cada ação e reação.

Pós-processamento: Explica o desenvolvimento do sistema de pixelização e contorno e sua influência no *pipeline* gráfico.

Interação do Usuário: Descreve a implementação dos elementos de interface gráfica e o sistema de mapeamento de entradas para suporte a múltiplos dispositivos, compondo a camada final de comunicação exibida sobre o viewport do jogo.

Som: Descreve os *softwares* utilizados e a implementação áudio espacial dos efeitos sonoros e música do jogo.

Configurações: Discorre sobre os aspectos técnicos que fazem parte da experiência do usuário e sobre as possíveis customizações do programa.

Planos Futuros: Apresenta ideias de melhorias e implementações que podem ser realizadas em versões futuras do jogo.

Conclusão: Resume os resultados obtidos, descreve as dificuldades encontradas e apresenta considerações finais sobre o trabalho.

Capítulo 2

Conceitos

2.1 Isometria

A projeção isométrica é um caso particular da projeção ortogonal que permite a representação gráfica tridimensional em superfícies bidimensionais. Matematicamente, a isometria é caracterizada pelo alinhamento dos eixos coordenados (**X**, **Y** e **Z**) em relação ao plano de projeção, de modo que os ângulos formados entre a projeção de cada par de eixos sejam exatamente iguais a 120°. Essa propriedade garante que a escala de medida permaneça uniforme ao longo de todos os eixos, eliminando os pontos de fuga característicos da perspectiva cônica, que é a forma como o olho humano enxerga a realidade.

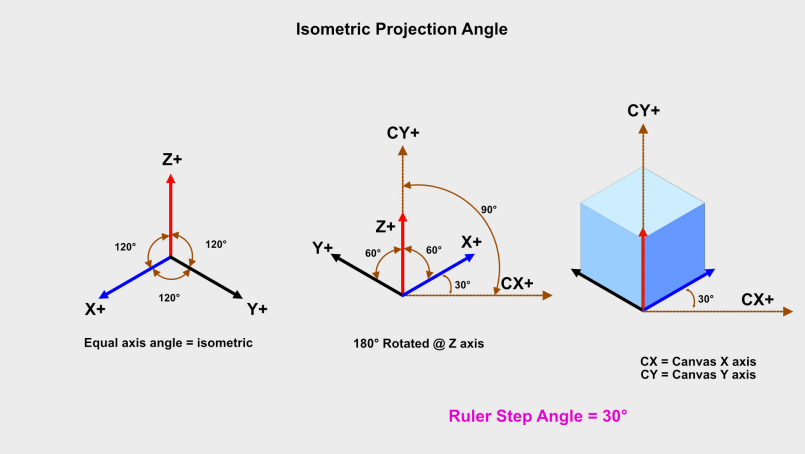


Figura 2.1: *Eixos X, Y e Z em uma projeção isométrica em relação ao canvas. Fonte:* RAFATPARVEZ, *2021*

Historicamente, o uso da perspectiva isométrica na indústria de jogos eletrônicos surgiu no início da década de 1980 como uma solução para contornar limitações computacionais dos hardwares da época. Antes do surgimento das unidades de processamento gráfico (GPUs) dedicadas ao 3D em tempo real, renderizar polígonos tridimensionais complexos era inviável. Os desenvolvedores passaram, então, a utilizar *sprites* bidimensionais desenhados

em ângulo isométrico, popularizando a clássica proporção de inclinação de 2:1 em *pixel art*, o que permitia criar uma ilusão de profundidade e volume sem o custo computacional de cálculos de perspectiva.

Todavia, esta proporção 2:1 não é uma projeção isométrica verdadeira. A projeção isométrica é um tipo de projeção axonométrica, que é uma categoria mais ampla de projeções ortogonais (paralelas). Nessas projeções, os ângulos entre os eixos podem variar, mas por serem projeções paralelas, elas eliminam os pontos de fuga característicos da perspectiva cônica. A projeção axonométrica é subdividida em três tipos:

Isométrica Caracteriza-se pela igualdade perfeita entre os ângulos formados pelos três eixos, cada um medindo exatamente 120° entre si (com uma inclinação de 30° em relação à linha horizontal de referência). Essa propriedade garante que a escala espacial e o fator de proporção sejam idênticos ao longo de todas as dimensões;

Dimétrica Apresenta dois ângulos iguais entre a projeção dos eixos e um ângulo distinto, resultando em duas escalas de medição idênticas e uma terceira diferente.

Trimétrica Todos os três ângulos entre os eixos são diferentes entre si.

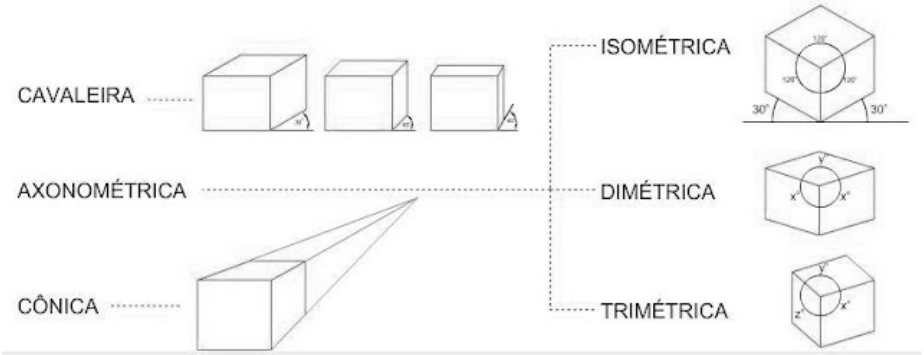


Figura 2.2: *Tipos de perspectivas mais utilizadas. Fonte:* Santos, 2017

Ou seja, em uma perspectiva isométrica perfeita (120°), a linha diagonal sobe a um ângulo exato de 30° ($\tan(30°) \approx 0{,}577$). Na era da *pixel art*, traçar uma linha com inclinação de 30° era impossível sem gerar *aliasing* (serrilhamento irregular) pesado na tela. Para resolver isso, os desenvolvedores criaram a regra prática de "andar" 2 *pixels* na horizontal para 1 *pixel* na vertical, resultando em uma inclinação de 26,565° ($\arctan(0{,}5) = 26{,}565°$), que é a proporção 2:1. Como essa inclinação não é exatamente 30°, os ângulos entre os eixos tornam-se aproximadamente 116,565°, 116,565° e 126,87°, o que caracteriza uma projeção dimétrica, mas que ainda é comumente referida como isométrica na indústria de jogos eletrônicos pois a simula muito bem visualmente.

Os principais marcos durante o pioneirismo desta técnica foram os jogos *Zaxxon* (1982), *Ant Attack* (1983) e *Knight Lore* (1984):

***Zaxxon* (1982):** É um jogo de tiro espacial onde o jogador pilota uma nave em uma base inimiga com movimento baseado em rolagem diagonal, sendo também um dos primeiros jogos a exibir sombras na tela, para indicar a posição da nave em relação à superfície. Desenvolvido pela SEGA em 1982, é amplamente reconhecido como

o primeiro jogo eletrônico a empregar a projeção axonométrica, que inclusive dá nome ao jogo (*AXXON* vem de *AXONométrica*).

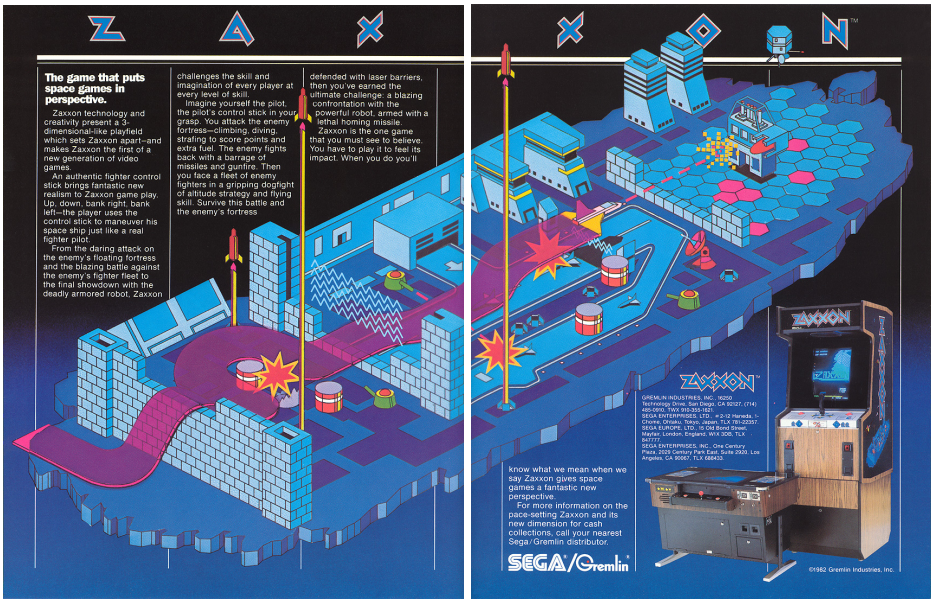


Figura 2.3: *Folheto promocional de Zaxxon para Arcade. Fonte: The Arcade Flyer Archive, 2026*

***Ant Attack* (1983):** Embora não tenha sido o primeiro jogo isométrico, foi o primeiro a adicionar um grau de liberdade vertical, permitindo que o jogador pudesse se mover livremente também para cima e para baixo, ao invés de apenas norte, sul, leste e oeste. Sendo talvez o primeiro jogo isométrico para computadores pessoais e também um dos primeiros a permitir que o jogador escolha seu gênero. Desenvolvido em 1983 por Sandy White para o ZX Spectrum, neste jogo o usuário começa escolhendo controlar o garoto ou a garota, então entra na cidade fortificada de Antescher para salvar o outro que foi capturado e imobilizado em algum lugar da cidade. A cidade é habitada por formigas gigantes que perseguem e mordem o jogador, cuja única forma de se defender é arremessando granadas que também podem machucar outros humanos. O objetivo do jogo é localizar o refém e escapar da cidade.

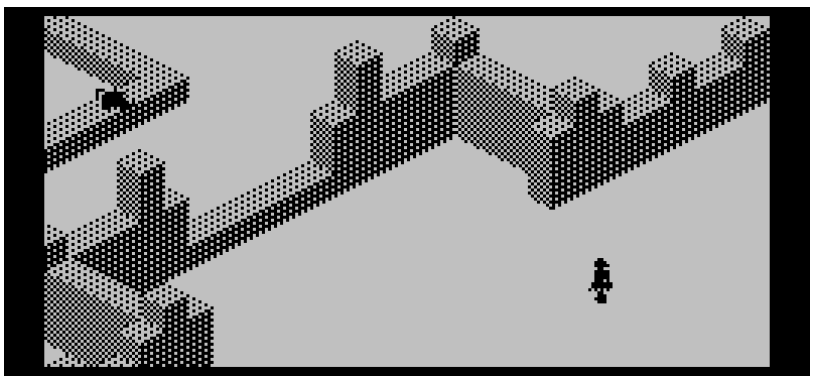


Figura 2.4: *Tela inicial de Ant Attack. Fonte: Arcade Idea, 2021*

***Knight Lore* (1984):** É um jogo de ação e aventura onde o jogador controla o herói *Sabreman* (da série que começou com *Sabre Wulf*), que foi mordido e amaldiçoado como lobisomem. Ele tem 40 dias para explorar um castelo de 128 salas, resolver quebra-cabeças e preparar um antídoto em um caldeirão. Foi um dos responsáveis por popularizar o gênero de aventura isométrica que por meio de sua técnica de mascaramento de imagem, chamada de *Filmation* (Garcia-Panella, 2010), permitia imagens passarem por baixo e por cima uma das outras sem que seus conteúdos colidissem, criando uma ilusão de profundidade que não era nativa dos computadores da época. O jogo foi publicado em 1984 e é considerado um trabalho seminal na história dos jogos britânicos, sendo coroado Jogo do Ano pela *Golden Joystick Awards* e pela *Popular Computing Weekly*, e lembrado como o primeiro jogo que ofereceu um mundo "explorável", cuja influência persiste em jogos de RPG para computador.

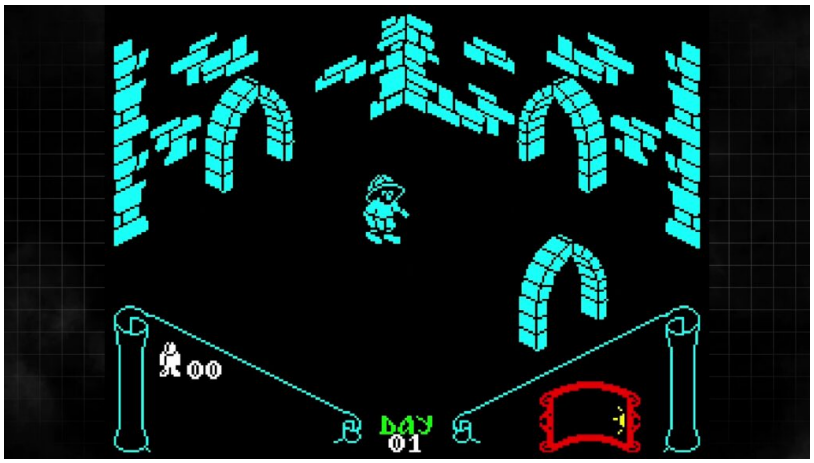


Figura 2.5: *Knight Lore. Fonte: Evercade, 2025*

No desenvolvimento moderno, a câmera isométrica é geralmente construída configurando-se uma câmera 3D com projeção ortográfica alinhada a um ângulo fixo. Do ponto de vista de *game design*, a adoção da câmera isométrica proporciona vantagens diretas para a jogabilidade e percepção do espaço, como:

Clareza Espacial e Legibilidade: Por não apresentar encurtamento perspectivo (*foreshortening*), os objetos mantêm a mesma escala visual independentemente da sua distância em relação à câmera. Isso facilita a leitura imediata de distâncias, trajetórias e posições no cenário.

Ampla Visão do Cenário: Permite que o jogador visualize simultaneamente o personagem principal, múltiplos inimigos ao redor e perigos do ambiente, favorecendo a antecipação de ações e o planejamento de reações.

Desempenho Visual Uniforme: A ausência de cálculo de deformação de perspectiva simplifica o *culling* de oclusão e otimiza a renderização de elementos estáticos e dinâmicos.

No entanto, o uso da isometria introduz desafios específicos que demandam soluções intencionais de design, como a oclusão de elementos (quando objetos altos do cenário

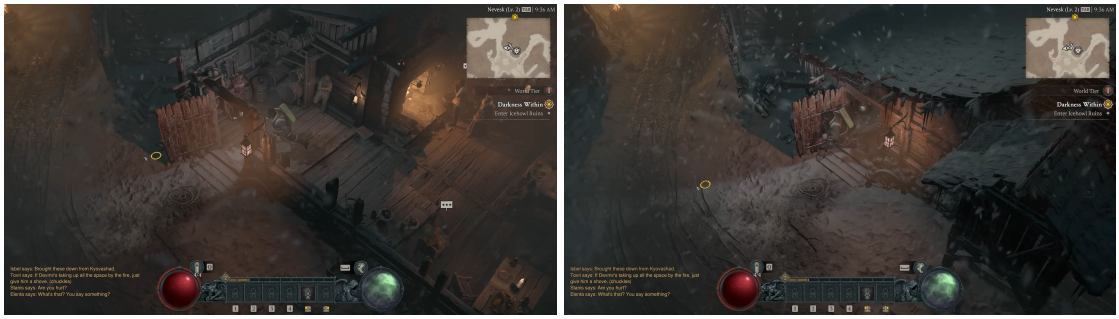
escondem o personagem ou inimigos atrás deles) e a percepção de altura em saltos. Essas particularidades exigem um planejamento de *level design* e o uso de *shaders* ou transparências dinâmicas para preservar a orientação do jogador durante sua navegação pelo mapa. Diferentes jogos resolvem problemas de isometria de maneiras distintas criativamente e tecnologicamente.

Efeito Fantasma Em jogos como Death's Door (2021) e Tunic (2022), quando o personagem passa atrás de uma árvore, parede ou ruína, o jogo utiliza um *shader* de recorte ou transparência que desenha uma silhueta do personagem "atrás" do obstáculo. Assim, o jogador continua enxergando exatamente onde seu personagem está, mesmo com a geometria física bloqueando a visão direta.

(a) *Death's Door (2021). Fonte: Acid Nerve, 2021* **(b)** *Tunic (2022). Fonte: TUNIC Team, 2022*

Figura 2.6: *É possível ver a silhueta dos personagens através dos obstáculos.*

Transparência Dinâmica Na presença de muitas paredes altas e masmorras como em Diablo IV (2023), quando o personagem se posiciona atrás de uma parede vertical em relação à câmera, um *shader* de transparência acionado por *raycasting*/linha de visão faz a parede desvanecer ou se dissolver completamente, revelando o espaço oculto.

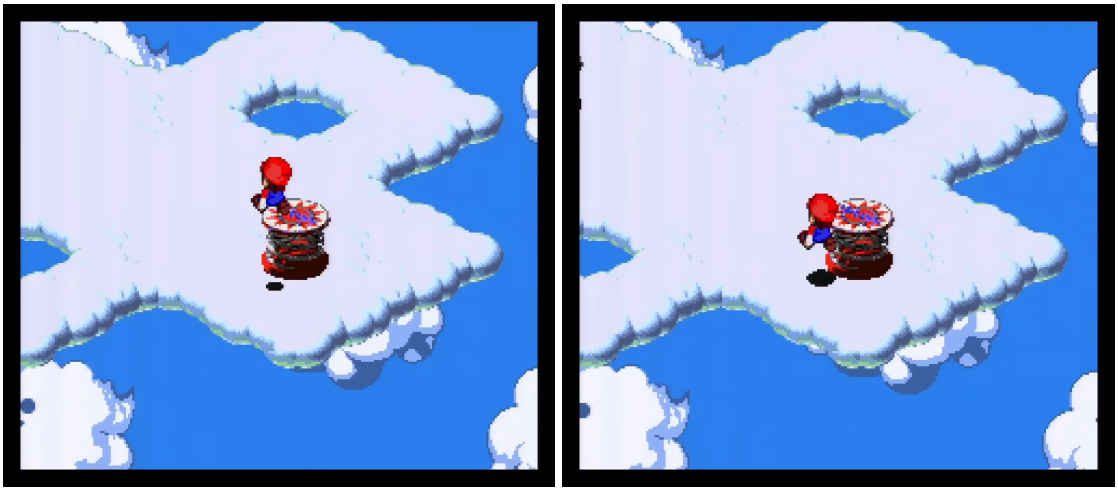


(a) *Visão interna da casa.* **(b)** *Visão externa da casa.*

Figura 2.7: *Transparência de obstáculos em Diablo IV (2023). Fonte: Blizzard Entertainment, 2023*

Sombras de Queda Os jogos isométricos utilizam da verticalidade para criar profundidade e complexidade espacial. Em jogos como Super Mario RPG (1996), a sombra do personagem não segue rigorosamente o ângulo da luz do sol do cenário. Em vez disso, há uma sombra projetada diretamente no plano do chão abaixo dele. Conforme

o personagem pula, a sombra permanece estática no solo, e a distância entre o personagem e sua sombra indica visualmente a sua altura atual.



(a) *Personagem em posição elevada.* **(b)** *Personagem em posição baixa.*

Figura 2.8: *Sombra de queda em Super Mario RPG (1996). Fonte: Square, 1996*

2.2 *Low Poly*

A identidade visual de um jogo 3D é fortemente influenciada pelo nível de detalhamento geométrico de seus modelos. O termo *low-poly* refere-se à prática de representar objetos tridimensionais utilizando um número reduzido de polígonos (primariamente triângulos e quadriláteros) em suas malhas poligonais (*meshes*).

Do ponto de vista da computação gráfica, a redução da contagem de vértices e polígonos diminui drasticamente o custo de processamento na etapa de transformação geométrica e rasterização da GPU. Historicamente adotada como solução obrigatória para garantir taxas de quadros aceitáveis em hardwares restritos, a modelagem *low-poly* se tornou, eventualmente, uma escolha artística comumente presente em jogos de desenvolvedores independentes devido a sua simplicidade e eficiência.

Para efeitos comparativos, nos anos 1990 o Playstation 1 prometia a renderização de até 360.000 polígonos por segundo em testes de laboratório, mas na prática os jogos renderizavam entre 90.000 a 180.000 polígonos por segundo devido ao custo de processamento de texturas, shaders, etc (VGCL: Video Game Console Library, 2026). Nesta época, um personagem costumava ter entre 400 a 800 polígonos dependendo de como o ambiente do jogo era renderizado (Jaybee, 2021). A Lara Croft, protagonista do jogo *Tomb Raider* (1996), possuía cerca de 230 polígonos em seu modelo original dentro de jogo (Croft Generation, 2011) (Embora algumas estimativas também citem valores como 540, dependendo se a fonte considera ou não partes do modelo como equipamentos ou outros acessórios promocionais (Robert Browne, 2014)).

Figura 2.9: *Lara Croft (1996). Fonte: Joshua Littlejohn, 2016*

Em *Tomb Raider: Underworld* (2008), já no Playstation 3, a quantidade de polígonos em seu modelo aumentou para 32,816, entrando para o *Guinness World Records* como a personagem de video-game mais detalhada da época (Guinness World Records, 2008).

Figura 2.10: *Lara Croft (2008). Fonte: Steam, 2008*

Em 2017, Johan Lithvall compartilhou sobre sua participação no desenvolvimento do jogo *Horizon Zero Dawn* (2017) para Playstation 4, onde ficou encarregado do cabelo da protagonista Aloy, e divulgou que ele possui cerca de 100.000 polígonos, mais que o triplo da quantidade de um personagem inteiro de 2008 (Johan Lithvall, 2017).

Figura 2.11: *Cabelo da Aloy (2017). Fonte: Johan Lithvall, 2017*

Os modelos *low poly* facilitam etapas do *pipeline* de produção como Mapeamento UV para as texturas, o *rigging* (estruturação do esqueleto dos modelos), e animação. pois com uma menor quantidade de polígonos, menor a necessidade de ponderações de peso, deformações e poses extremamente complexas, simplificando os movimentos dos personagens.

No contexto artístico, o *low-poly* se destaca por sua abstração geométrica, pois ao dispensar texturas de alta resolução e malhas densas (*high-poly*), os artistas utilizam a própria estrutura dos polígonos para definir volumes, silhuetas e superfícies. Essa abordagem é frequentemente combinada com técnicas de sombreamento plano (*flat shading*), onde as normais de iluminação são calculadas por face e não por vértice, acentuando as arestas e proporcionando um aspecto visual limpo e estilizado.

Figura 2.12: *Sombreamento plano em Virtua Racing (1992). Fonte: Moby Games, 2014*

Além disso, a união de modelos *low-poly* com texturas pixelizadas (que é a proposta

adotada neste projeto) gera uma atmosfera estética que se assemelha aos jogos eletrônicos clássicos, reinterpretando-a com ferramentas tridimensionais modernas. Essa linguagem visual não apenas reforça a legibilidade espacial exigida por dinâmicas de jogo aceleradas, mas também reduz o consumo de memória de vídeo (VRAM), permitindo que a capacidade de processamento seja direcionada para a estabilidade da física e para a fluidez das interações no cenário.

2.3 *Fast-Paced*

Diferente de jogos focados em exploração de paisagens ou turnos estratégicos, os jogos *fast-paced* priorizam a velocidade e exigem do jogador respostas motoras rápidas, reflexos apurados e tomadas de decisão instantâneas em meio a sequências de ações.

Sob a ótica do *game design*, o desenvolvimento de um jogo *fast-paced* impõe desafios à engenharia de movimentação e à arquitetura de colisão. Elementos como aceleração, desaceleração, inércia e transições de animação precisam ser ajustados para eliminar qualquer sensação de atraso (*input lag* ou lentidão de resposta). O personagem deve responder de forma imediata aos comandos do usuário, permitindo manobras fluidas como deslizes por superfícies, saltos e desvios de projéteis.

Além disso, o ritmo do jogo está diretamente atrelado à clareza do *level design*. Em cenários dinâmicos, o jogador não pode perder tempo tentando entender o espaço ao seu redor. A arquitetura dos mapas e os elementos visuais devem comunicar instantaneamente as rotas de fuga, os perigos e a posição dos inimigos.

Assim, a combinação da estética *low-poly* pixelizada e a projeção isométrica atuam de forma sinergética com o gênero, pois ao remover ruídos visuais complexos e texturas fotorrealistas pesadas, a cena torna-se limpa, garantindo a legibilidade espacial necessária para que o jogador possa agir e reagir de forma fluida e sem comprometer o desempenho computacional. Tal combinação é exemplificada na série de jogos *Toree* desenvolvidos pela *Siactro* — nome profissional do desenvolvedor independente Marcus Horn — que são jogos de plataforma curtos com estética inspirada na era do PlayStation 1 e Nintendo 64. O objetivo é correr o mais rápido possível por fases lineares cheias de armadilhas, mantendo o multiplicador de velocidade no máximo.

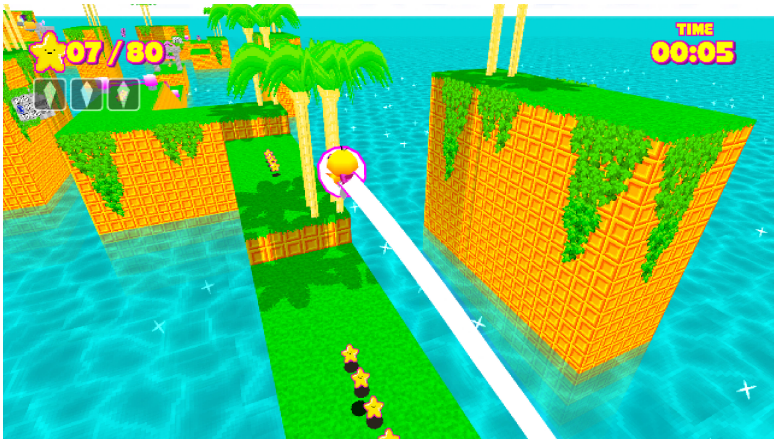


Figura 2.13: *Toree Genesis (2022). Fonte: SIACTRO, 2022*

Capítulo 3

Metodologia

Devido a natureza do projeto em grupo, foram adotadas diversas convenções de documentação, controle de versão e sistemas de gerenciamento para conservar o fluxo de desenvolvimento.

3.1 Projeto

O jogo foi desenvolvido utilizando o software Godot, que é uma ferramenta de desenvolvimento de jogos. Foi utilizada sua própria linguagem (GDScript), que é uma linguagem de alto-nível e orientada a objetos, com uma sintaxe similar a python. O objetivo dessa linguagem é ser otimizada e fortemente integrada à Godot Engine, permitindo maior flexibilidade de criação e integração. Para manter o código-fonte organizado, e legível para todos os integrantes, foi adotada a convenção de estilo oficial do GDScript, que pode ser encontrada em sua documentação.

Para gerenciar as versões do projeto, foi criado um repositório no Github, onde foram adotadas e adaptadas as convenções semânticas estabelecidas em **padroes-de-commit** (https://github.com/iuricode/padroes-de-commits), um repositório brasileiro com quase 10 mil estrelas. O padrão descrito por esse grupo descreve um *commit* como:

```
:emoji: tipo(escopo opcional): descrição
```

emoji: Representa o *commit* visualmente de acordo com a tabela de emojis estabelecida. É recomendado, porém não é obrigatório.

tipo: Representa o tipo do *commit*, como *feat* ou *fix*.

escopo: Representa o escopo do *commit*, como *ui* ou *gameplay*.

!: Opcional. Indica que o *commit* é uma mudança que quebra a compatibilidade com versões anteriores. Como em *feat(ui)!: change button style*.

descrição: Resumo curto do que foi alterado no *commit*. Deve ser escrito no imperativo.

Para este projeto não foram utilizados os fatores opcionais, portanto os *commits* seguiram os seguintes exemplos de formatação:

```
fix: update fallback paths for player animations
feat: add navmeshes to stage platforms
refactor: move animation_player initialization to Character class
```

Mantendo desta forma os *commits* concisos, sem necessidade de descrições detalhadas para cada um.

A *branch main* foi estabelecida como uma *branch* privada, então não é possível realizar *commits* ou *merge* nela sem permissão do dono do repositório. Desta forma, a *branch* principal do projeto recebe apenas códigos completos e funcionais.

3.2 Desenvolvimento

Para o fluxo de desenvolvimento foi aplicado o método ágil *Kanban*, onde cada tarefa foi separada em **Planejado**, **Desenvolvimento**, **Testes**, **Completo**. Quando no estado **Completo** a tarefa era avaliada por outro integrante para uma segunda opinião e só depois de ser aceita na branch main que era removida da lista de tarefas. As tarefas eram definidas em reuniões online esporádicas, troca de mensagens ou encontros presenciais onde os integrantes do grupo se ajudavam em alguma área complexa ou que estavam tendo dificuldades, por meio de *pair programming* ou *brainstorm*.

Para manter esse controle foi utilizado o *software* Obsidian, uma ferramenta de organização e anotação de projetos. Foram criados dois templates neste *software*: **Devlogs** e **Tasks**. Os **Devlogs** contêm anotações rápidas sobre o que foi feito, quando e por quem, referências utilizadas como tutoriais, documentações e artigos, e a opção de atrelar uma **Task** (tarefa) ao *devlog* daquele dia. Cada **Task** contém uma descrição simples e concisa que pode ser subdividida em várias etapas que compõem a tarefa. Cada tarefa pode ser atribuída a uma ou mais pessoas.

3.3 Arquitetura

O código-fonte, os recursos artísticos e a documentação do projeto foram estruturados de forma modular, visando a separação de responsabilidades e a manutenibilidade do *software*. A arquitetura de diretórios do repositório está dividida em quatro pastas principais: **docs**, **assets**, **src** e **util**, conforme descrito a seguir:

docs/: Concentra toda a documentação gerada ao longo do projeto, incluindo o *site*, a monografia acadêmica e os arquivos de apresentação.

assets/: Agrega todos os arquivos de recursos estáticos consumidos pela *engine* e pelos *scripts*. É subdividido em elementos como modelos tridimensionais, malhas poligonais, texturas, animações, fontes tipográficas, partículas, diálogos estruturados, arquivos de imagem e componentes gráficos de HUD.

util/: Contém funções auxiliares de propósito geral e ferramentas de suporte reutilizáveis por diferentes módulos do sistema, promovendo a reutilização de código e a redução de acoplamento.

src/: Representa o núcleo lógico do sistema, contendo o código-fonte responsável pelo comportamento do jogo. A estrutura interna divide-se em subdomínios como:

- **Áudio**: Implementação e reprodução de efeitos sonoros e música.
- **Entidades**: Dividido em *scripts* de lógica de controle do jogador, interações físicas, implementação de objetos instanciáveis e máquina de estados dos inimigos e do jogador.
- **GUI**: Gerenciamento de recursos de interface.
- **Globais**: *Scripts* de gerenciamento do jogo, estado global e constantes do sistema.
- **Shaders**: Programas executados na GPU para efeitos visuais e pós-processamento.
- **Estágios**: Cenas, *scripts* de *level design* e lógica de carregamento dos mapas.
- **Terrenos**: Cenas que compõem as cenas dos estágios.

3.4 Modelagem

A criação dos modelos tridimensionais do projeto — incluindo personagens, cenários, itens e entidades interativas — foi feita utilizando os softwares Blender e Blockbench. A escolha de múltiplas ferramentas permitiu alocar cada *software* à sua respectiva especialidade técnica dentro da estética *low-poly* pixelizada.

O Blender foi empregado na construção de geometrias mais complexas, modelagem orgânica e ajustes estruturais avançados de malhas poligonais. Em contrapartida, o Blockbench foi adotado como a principal ferramenta para a criação dos modelos low-poly baseados em formas simples e modelagem modular. O Blockbench destaca-se por sua compatibilidade nativa com a estética estilizada de baixa poligonia e alinhamento de *pixel art*, facilitando o mapeamento de texturas por pixel (*texture mapping*).

As animações dos personagens e entidades foram desenvolvidas diretamente no ambiente de animação do Blockbench. Para transformá-los em corpos articulados, foi utilizado um sistema de hierarquia óssea (*rigging e skeletal animation*). Nesse sistema, as malhas poligonais são vinculadas a nós de articulação (*bones*), que podem então ser controlados durante a animação para fazer os personagens se moverem.

O fluxo de animação consistiu na definição de sequências baseadas em *keyframes* (quadros-chave) para transições de estados, tais como locomoção, deslize (*slide*), ataque e morte. Como o estilo visual prioriza a legibilidade de silhuetas e o dinamismo *fast-paced*, as animações foram projetadas com quadros de antecipação e recuperação curtos, assegurando que o *feedback* visual acompanhasse a resposta imediata dos comandos do jogador.

Para integrar os modelos e animações gerados no Blender e no Blockbench à engine de desenvolvimento, utilizou-se o formato aberto glTF (*GL Transmission Format*) em sua extensão binária, o arquivo .glb.

O formato .glb foi escolhido por sua eficiência como um contêiner autocontido. Diferente de formatos antigos que exigem múltiplos arquivos separados para malhas, materiais, texturas e esqueletos de animação, o arquivo binário .glb compacta toda essa estrutura em um único pacote otimizado. Isso garantiu que, no momento da importação para o motor do jogo, as malhas tridimensionais, os dados de esqueleto, as curvas de animação e as informações de mapeamento estivessem unificados e prontos para renderização, preservando a integridade visual e reduzindo o tempo de carregamento dos recursos do jogo.

3.5 *Shaders*

Os efeitos de renderização gráfica, estilização visual e tratamento de materiais do projeto foram implementados utilizando o subsistema de *shaders* do motor gráfico Godot. Os *shaders* são programas executados em paralelo na Unidade de Processamento Gráfico (GPU), dividindo-se fundamentalmente em duas categorias no pipeline de renderização:

***Vertex Shaders* (*Shaders* de Vértice)** Executados para cada vértice da malha tridimensional, são responsáveis por transformar as coordenadas espaciais dos objetos (do espaço local para o espaço de projeção) e permitir manipulações geométricas diretas.

***Fragment*/*Pixel Shaders* (*Shaders* de Fragmento)** São executados na etapa de rasterização, que é o processo de conversão de uma imagem vetorial (composta de vetores e outros dados geométricos) em uma imagem raster (formato de imagem que contêm a descrição de cada pixel ou pontos). São encarregados de calcular a cor, o sombreamento, as propriedades de textura e a opacidade de cada pixel renderizado na tela.

Capítulo 4

Proposta

4.1 Câmera

A visualização do ambiente de jogo adota uma perspectiva tridimensional com projeção ortográfica, configurada sob uma inclinação espacial de estilo isométrico, focando na legibilidade espacial dos cenários.

Para auxiliar nesta legibilidade, diferente de abordagens tradicionais com ângulos estáticos de visualização, o sistema de câmera do projeto implementa um grau de liberdade de rotação horizontal contínua em 360° ao redor do personagem principal. Essa rotação permite ao jogador girar o ponto de vista livremente para examinar cantos ocultos do mapa, antecipar ameaças e navegar por rotas complexas sem alterar a inclinação vertical fixa da projeção. Esta técnica é comumente encontrada em CRPGs (*Computer Role-Playing Games*) isométricos modernos, como *Divinity: Original Sin 2* (Larian Studios, 2017).

Para assegurar o ritmo acelerado (*fast-paced*) da jogabilidade, o acompanhamento da câmera em relação ao movimento do jogador é projetado para ocorrer de forma suave (*smooth follow*). Esse amortecimento visual evita saltos bruscos resultantes de mudanças de direção repentinas do personagem, mantendo a cena estável.

Por fim, o comportamento da câmera foi especificado para lidar com os desafios de oclusão inerentes à perspectiva isométrica. Ao posicionar o jogador atrás de estruturas altas ou elementos volumosos do cenário, a câmera e os sistemas visuais integrados devem garantir a manutenção do *feedback* de localização do personagem, preservando a orientação espacial e a fluidez dos comandos em qualquer direção do mapa.

4.2 Movimentação

O sistema de locomoção do personagem principal foi estruturado para atender aos requisitos de alta velocidade e resposta imediata inerentes do gênero *fast-paced*. O controle de deslocamento ocorre de maneira omnidirecional no plano horizontal do cenário, permitindo ao jogador mover-se fluidamente em qualquer direção independentemente da rotação atual da câmera.

Para garantir o dinamismo exigido pelo projeto, o conjunto de ações fundamentais de movimento é composto em:

Deslocamento: O personagem responde de forma contínua aos comandos direcionais.

Corrida: O jogador dispõe de um estado de corrida que aumenta a sua velocidade de locomoção pelo mapa.

Pulo: Mecânica de salto para permitir a navegação vertical sobre as plataformas sem quebrar o fluxo contínuo de movimento.

Do ponto de vista de *game design*, as transições entre estados de movimento (como a entrada e saída da corrida ou a execução de saltos) foram desenvolvidas com foco na conservação de momento. Em vez de travamentos ou freadas abruptas, o sistema de movimento preserva a energia cinética acumulada pelo jogador. Isto permite, por exemplo, que a velocidade obtida durante uma corrida ou um deslize seja transferida para o pulo. Essa continuidade física assegura que o jogador mantenha total controle e fluidez ao se locomover pelo ambiente.

4.3 Progressão e Mecânicas de Jogo

O jogo estrutura-se em uma progressão linear de três fases, onde cada fase é composta por um ambiente diferente que introduz novos desafios espaciais e concede ao jogador uma nova habilidade por meio de um item, expandindo tanto suas opções de exploração quanto suas opções de combate.

4.3.1 Fase 1 - Montanha (Mobilidade e Defesa)

Ambiente externos de alta inclinação, nesta fase é possível obter o primeiro item, a **Tampa**. Também é introduzido o primeiro inimigo — o **Besouro**, que atira projéteis ácidos em direção ao jogador. Utilizando a tampa é possível ter acesso a duas mecânicas do jogo:

Deslizar: O protagonista utiliza a tampa como prancha para deslizar em superfícies inclinadas, podendo se mover mais rápido ou gerar velocidade para alcançar áreas que não são alcançadas normalmente.

Bloquear: É possível utilizar a tampa de escudo para se defender dos projéteis conjurados pelos besouros a distância.

4.3.2 Fase 2 - Caverna (Interação Física e Ataque)

Um ambiente fechado e complexo que testa a capacidade de exploração do jogador. Nele, é obtido o segundo item, a **Tesoura**. O inimigo apresentado nesta região é a **Formiga**, que persegue o jogador para mordê-lo. Com a tesoura, são aprendidas duas novas mecânicas:

Empurrar: Realizando um movimento de estocada com a tesoura, o personagem consegue interagir com elementos do cenário (como empurrar pilares) para abrir novos caminhos ou criar meios de se atingir um local.

Bater: Usando a tesoura de espada, o protagonista consegue bater nos besouros e nas formigas para tentar eliminá-los e facilitar sua jornada.

4.3.3 Fase 3 - Floresta (Controle Tático e Quebra-cabeças)

Ambiente escuro e misterioso que desafia o raciocínio lógico do jogador. Explorando esse ambiente, o jogador encontra o terceiro item, o **Vaga-lume**. Nesta região é introduzido o terceiro inimigo — o **Tatu-bolinha**, que, ao ver o jogador, começa a rolar em sua direção, e quando entra em contato com alguma superfície enquanto neste estado, ele explode. Com a ajuda do vaga-lume, é possível utilizar agora também as duas últimas mecânicas:

Iluminar:

Paralizar:

4.4 Efeitos Visuais

4.5 Pós-processamento

4.6 Interação do Usuário

4.7 Som

4.8 Configurações

Capítulo 5

Entidades

Capítulo 6

Planos Futuros

Capítulo 7

Conclusão

Referências

[Acid Nerve 2021] Acid Nerve. *Death's Door*. 2021. URL: https://store.steampowered.com/app/894020/Deaths_Door/ (citado na pg. 9).

[Arcade Idea 2021] Arcade Idea. *Ant Attack (1983)*. 5 de abr. de 2021. URL: https://arcadeidea.wordpress.com/2021/04/05/ant-attack-1983/ (citado na pg. 7).

[Blizzard Entertainment 2023] Blizzard Entertainment. *Diablo IV*. 2023. URL: https://store.steampowered.com/app/2344520/Diablo_IV/ (citado na pg. 9).

[Croft Generation 2011] Croft Generation. *Lara Croft*. 12 de mai. de 2011. URL: https://croftgeneration.fandom.com/wiki/Lara_Croft (citado na pg. 10).

[Evercade 2025] Evercade. *Knight Lore (1984)*. 11 de dez. de 2025. URL: https://evercade.co.uk/evercade-game-spotlight-knight-lore/ (citado na pg. 8).

[Garcia-Panella 2010] Oscar Garcia-Panella. *Knight Lore and the third dimension*. Berlin, Heidelberg: Springer-Verlag, 2010. ISBN: 9780557844517 (citado na pg. 8).

[Guinness World Records 2008] Guinness World Records. *Most detailed videogame character*. 18 de nov. de 2008. URL: https://www.guinnessworldrecords.com/world-records/92723-most-detailed-videogame-character (citado na pg. 11).

[Jaybee 2021] Jaybee. *An Overview of PlayStation Aesthetics*. 6 de abr. de 2021. URL: https://www.patreon.com/jaybeemadethis/posts/overview-of-49375996 (citado na pg. 10).

[Johan Lithvall 2017] Johan Lithvall. *Horizon Zero Dawn - Hair*. 5 de mai. de 2017. URL: https://www.artstation.com/artwork/EDbk4 (citado nas pgs. 11, 12).

[Joshua Littlejohn 2016] Joshua Littlejohn. *Tomb Raider 20th Anniversary: The Evolution of Lara Croft*. 15 de nov. de 2016. URL: https://www.fandom.com/articles/tomb-raider-20th-anniversary-evolution-lara-croft-20-year-celebration (citado na pg. 11).

[Larian Studios 2017] Larian Studios. *Divinity: Original Sin 2 - Definitive Edition*. 2017. URL: https://store.steampowered.com/app/435150/Divinity_Original_Sin_2_Definitive_Edition/ (citado na pg. 19).

[Moby Games 2014] Moby Games. *Virtua Racing*. 26 de jan. de 2014. URL: https://www.mobygames.com/game/8131/virtua-racing/screenshots/arcade/654788/ (citado na pg. 12).

[rafatparvez 2021] rafatparvez. *How to do Isometric Art/ illustration*. 8 de mai. de 2021. URL: https://tips.clip-studio.com/en-us/articles/4795 (citado na pg. 5).

[Robert Browne 2014] Robert Browne. *The Many Faces of Lara Croft: Tomb Raider [Infographic]*. 20 de jan. de 2014. URL: https://www.halloweencostumes.com/blog/p-468-tomb-raider-infographic.aspx (citado na pg. 10).

[Santos 2017] Rafael Nascimento dos Santos. *Perspectivas: Tipos Utilizados no Curso de Arquitetura*. 3 de mar. de 2017. URL: https://www.arquitetoversatil.com/2017/03/perspectivas-tipos-utilizados-no-curso-de-arquitetura.html (citado na pg. 6).

[Siactro 2022] Siactro. *Toree Genesis*. 2022. URL: https://siactro.itch.io/toree-genesis (citado na pg. 14).

[Square 1996] Square. *Super Mario RPG: Legend of the Seven Stars*. 1996. URL: https://www.ign.com/games/super-mario-rpg-legend-of-the-seven-stars (citado na pg. 10).

[Steam 2008] Steam. *Tomb Raider: Underworld*. 21 de nov. de 2008. URL: https://store.steampowered.com/app/8140/Tomb_Raider_Underworld/ (citado na pg. 11).

[The Arcade Flyer Archive 2026] The Arcade Flyer Archive. *Zaxxon (1982) Flyer*. 2026. URL: https://flyers.arcade-museum.com/videogames/show/2535 (citado na pg. 7).

[TUNIC Team 2022] TUNIC Team. *Tunic*. 2022. URL: https://store.steampowered.com/app/553420/TUNIC/ (citado na pg. 9).

[VGCL: Video Game Console Library 2026] VGCL: Video Game Console Library. *Sony PlayStation (PS1)*. 15 de set. de 2026. URL: https://www.videogameconsolelibrary.com/console/sony-playstation-ps1/ (citado na pg. 10).